

Simulador de Comportamiento Vial con Inteligencia Artificial

SBVIA

Administración de Bases de Datos – Examen Final

Integrantes:

-  Justyn K. Cruz Pérez
-  Jefferson M. Umaginga Arévalo
-  Diego A. Zamora Bumbila

Docente:

Ing. Cordero Bazurto José Steven, SOFT-R

Motor de BD:

 PostgreSQL 16

Fecha: Septiembre 2026

Agenda

- 1 Problema y Objetivos
- 2 Arquitectura del Sistema
- 3 Modelado de la Base de Datos
- 4 Demostración de la Aplicación
- 5 Carga Masiva de Datos
- 6 Control de Usuarios, Roles y Permisos
- 7 Auditoría
- 8 Respaldos
- 9 Optimización e Índices
- 10 Conclusiones

El Problema que Resuelve SBVIA



Problema identificado

- Los conductores novatos tienen **exposición limitada** a escenarios de riesgo antes de obtener la licencia.
- La formación tradicional **no registra** decisiones ni métricas de desempeño.
- Difícil identificar **patrones de error** repetitivos sin evidencia digital.



Solución: SBVIA

- Entorno **interactivo y controlado** para práctica de decisiones viales.
- **Registro automático** de cada acción, infracción y métrica.
- **Retroalimentación inmediata** del sistema e IA.
- Base de datos **completamente auditable** en PostgreSQL 16.

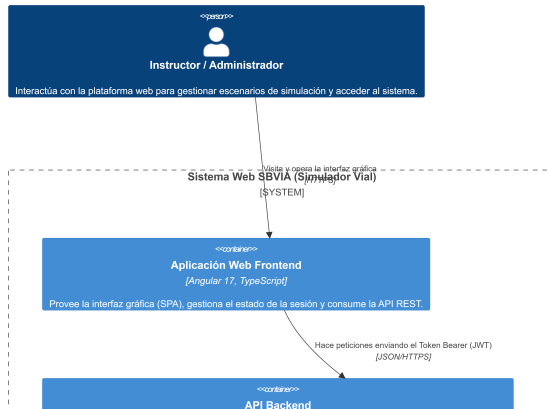


Objetivo del proyecto

Desarrollar y validar una plataforma web reproducible que permita entrenar y evaluar conductores mediante escenarios virtuales, con una base de datos PostgreSQL correctamente administrada, auditada y optimizada.

Arquitectura General del Sistema

Diagrama C4 Nivel 2 (Contenedores) - Plataforma SBVIA



Tablas (21 total)

Catálogos (12):

rol, estado_usuario, estado_simulacion, tipo_via, tipo_clima,
nivel_dificultad, nivel_gravedad, tipo_evento, tipo_metrica,
tipo_vehiculo, modelo_ia, regla_transito

Transaccionales (9):

usuario, vehiculo, escenario, sesion_entrenamiento, simulacion,
evento_vial, decision, infraccion, historial_acceso ...

Restricciones e integridad

- **14 claves foráneas** con ON UPDATE CASCADE
- **20 restricciones CHECK** a nivel de motor
- **Constraints UNIQUE** en correo, nombre_usuario, escenario
- **GENERATED BY DEFAULT AS IDENTITY** en todas las PKs

Vistas del sistema (5)

vw_historial_simulaciones, vw_infracciones_detalladas, vw_metricas_simulacion, vw_ranking_participantes,
vw_retroalimentacion_usuario

Ejemplo de Tabla con Restricciones

Listing 1: Tabla usuario – PKs, FKs y CHECK

```
1 CREATE TABLE public.usuario (  
2     id_usuario      bigint NOT NULL,          -- PK IDENTITY  
3     nombres         varchar(100) NOT NULL,  
4     correo          varchar(150) NOT NULL,     -- UNIQUE  
5     contrasena_hash varchar(255) NOT NULL,  
6     intentos_fallidos integer DEFAULT 0 NOT NULL,  
7     cuenta_bloqueada boolean DEFAULT false NOT NULL,  
8     id_rol          integer NOT NULL,         -- FK -> rol  
9     id_estado_usuario integer NOT NULL,       -- FK -> estado_usuario  
10    CONSTRAINT chk_usuario_correo  
11        CHECK (correo ~* '^[A-Z0-9-_%+~]+@[A-Z0-9-]+\.[A-Z]{2,}$'),  
12    CONSTRAINT chk_usuario_contrasena_hash  
13        CHECK (length(btrim(contrasena_hash)) >= 20)  
14 );
```

Relación simulacion (tabla central)

simulacion $\leftarrow$ sesion_entrenamiento $\leftarrow$ usuario
simulacion $\rightarrow$ escenario, vehiculo, estado_simulacion
simulacion $\leftarrow$ evento_vial, decision, infraccion, metrica_desempeno

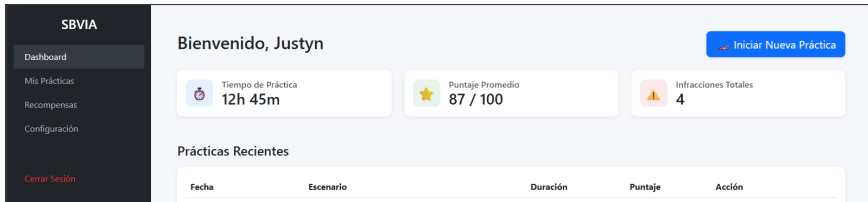
Módulos del Sistema SBVIA

Participante (ROLE_PARTICIPANTE)

- Registro e inicio de sesión seguro (JWT + cookie)
- Selección de escenarios viales
- Ejecución de simulaciones con eventos y decisiones
- Consulta de su propio historial y métricas
- Retroalimentación automática del sistema

Administrador (ROLE_ADMIN)

- Gestión de usuarios (crear, editar, bloquear)
- Administración de escenarios y vehículos
- Visualización del historial de accesos (auditoría)
- Generación de reportes de actividad
- Acceso al ranking de participantes



The screenshot shows the SBVIA user interface. On the left is a dark sidebar with the SBVIA logo and navigation links: Dashboard, Mis Prácticas, Recompensas, Configuración, and Cerrar Sesión. The main content area is light blue and greets the user as 'Bienvenido, Justyn'. It features three summary cards: 'Tiempo de Práctica' (12h 45m), 'Puntaje Promedio' (87 / 100), and 'Infracciones Totales' (4). Below these is a section for 'Prácticas Recientes' with a table header containing Fecha, Escenario, Duración, Puntaje, and Acción. A blue button 'Iniciar Nueva Práctica' is located in the top right corner.

Millón de Registros – Estrategia

Script:

01_generar_millon_registros.sql

- Usa generate_series() de PostgreSQL
- Desactiva triggers con SET session_replication_role = 'replica' para velocidad
- Genera datos **coherentes y relacionados**
- Ejecuta VACUUM ANALYZE al finalizar

```
1 INSERT INTO usuario (nombres, correo, ...)
2 SELECT 'Nombres_' || i,
3        'usr' || i || '@sbvia.edu.ec',
4        ...
5 FROM generate_series(1, 2000) AS i;
```

Distribución de registros

Tabla	Registros
decision	600,000
progreso_simulacion	300,000
evento_vial	150,000
historial_acceso	100,000
infraccion	60,000
simulacion	30,000
otras tablas	132,000
TOTAL	≈1,372,000

Listing 2: Consulta de verificación (02_verificar_millon_registros.sql)

```
1 SELECT 'decision'          AS tabla, COUNT(*) AS registros FROM decision
2 UNION ALL SELECT 'progreso_simulacion', COUNT(*) FROM progreso_simulacion
3 UNION ALL SELECT 'evento_vial', COUNT(*) FROM evento_vial
4 UNION ALL SELECT 'historial_acceso', COUNT(*) FROM historial_acceso
5 UNION ALL SELECT 'simulacion', COUNT(*) FROM simulacion
6 ORDER BY registros DESC;
7
8 -- Tamaño total de la base de datos:
9 SELECT pg_size_pretty(pg_database_size(current_database())) AS tamano_bd;
```

Evidencia en vivo – DBeaver

Ejecutar la consulta anterior en DBeaver y mostrar el resultado
(El total debe superar 1,000,000 de filas distribuidas en múltiples tablas)

Control de Usuarios y Roles – Dos Niveles

Nivel 1: Roles de aplicación (tabla rol)

Rol	Acceso
ADMINISTRADOR	Total
INSTRUCTOR	Escenarios + reportes
PARTICIPANTE	Solo su historial

Controlado por Spring Security + JWT en cada endpoint REST.

Respuestas diferenciadas: **401** (sin token) / **403** (sin permiso)

Nivel 2: Usuarios PostgreSQL (BD)

Usuario DB	Permisos
sbvia_admin	Todo (DDL + DML)
sbvia_app	SELECT/INSERT/UPDATE/DELETE
sbvia_readonly	Solo SELECT

Script: 03_usuarios_roles_permisos.sql



Tabla: historial_acceso

Registra **cada intento de login**:

- fecha_hora, direccion_ip
- dispositivo, navegador
- acceso_exitoso (bool)
- detalle del evento

100,000+ registros en la BD actual

Trigger automático

trg_actualizar_ultimo_acceso

AFTER INSERT en historial_acceso:

- Éxito → actualiza ultimo_acceso
- Fallo → incrementa intentos_fallidos

Listing 3: Consulta de auditoría

```
1  -- Accesos ultimas 24h
2  SELECT
3      concat(u.nombres, ' ', u.apellidos)
4      AS usuario,
5      ha.fecha_hora,
6      ha.direccion_ip,
7      ha.acceso_exitoso
8  FROM historial_acceso ha
9  JOIN usuario u
10     ON u.id_usuario = ha.id_usuario
11  WHERE ha.fecha_hora >=
12         NOW() - INTERVAL '24 hours'
13  ORDER BY ha.fecha_hora DESC;
```

Respaldos con pg_dump



Generación del respaldo

```
1  -- Respaldo completo (formato custom)
2  pg_dump \
3    --host=localhost --port=5432 \
4    --username=sbvia_admin \
5    --dbname=sbvia \
6    --format=custom \
7    --compress=9 \
8    --file=sbvia_20260908.backup
9
10 -- Desde Docker:
11 docker compose exec db pg_dump \
12   -U sbvia_user -d sbvia -Fc \
13   -f /backups/sbvia.backup
14
```



Restauración

```
1  -- Crear BD destino
2  createdb -U sbvia_admin sbvia_rest
3
4  -- Restaurar
5  pg_restore \
6    -U sbvia_admin \
7    -d sbvia_rest \
8    --verbose --clean \
9    sbvia_20260908.backup
10
11 -- Verificar
12 SELECT COUNT(*) FROM simulacion;
13
```

Evidencia

Archivo database/current/schema.sql
62,888 bytes – dump del esquema completo con

Índices Implementados (13 índices B-Tree)

Índices principales

Índice	Tabla
idx_simulacion_usuario_fecha	simulacion
idx_simulacion_escenario	simulacion
idx_simulacion_estado	simulacion
idx_decision_simulacion_fecha	decision
idx_evento_simulacion_fecha	evento_vial
idx_historial_acceso_usuario_fecha	historial_acceso
idx_metrica_simulacion_tipo	metrica_desempeno
...	...

Consulta crítica identificada

Historial de simulaciones de un usuario – ejecutada en **cada carga del dashboard**:

- Filtra por `id_usuario`
- Ordena por `fecha_inicio DESC`
- JOIN con `escenario`, `vehículo`, `estado`
- Sobre tabla con **30,000+ filas**

Justificación del índice

(`id_usuario`, `fecha_inicio DESC`) cubre el filtro WHERE y el ORDER BY en una sola operación de índice

EXPLAIN ANALYZE – Antes y Después

✗ SIN índice – Sequential Scan

```
1 Seq Scan on simulacion
2   Filter: (id_usuario = 42)
3   Rows Removed by Filter: 29985
4   cost=0.00..12148.00
5   actual time=165.221..165.310
6   rows=15 loops=1
7
8 Execution Time: 284.5 ms
9
```

Lee las **30,000** filas completas, descarta 29,985

✓ CON índice – Index Scan

```
1 Index Scan using
2   idx_simulacion_usuario_fecha
3   on simulacion
4   Index Cond: (id_usuario = 42)
5   cost=0.29..12.38
6   actual time=0.031..0.188
7   rows=15 loops=1
8
9 Execution Time: 0.52 ms
10
```

Lee **solo 15** filas directamente del índice

0.52 ms

284.5 ms (sin índice)

≈ **546**× más rápido



Aprendizajes técnicos

- PostgreSQL ofrece mecanismos robustos de integridad declarativa (CHECK, FK, UNIQUE) que reducen la dependencia de validaciones en la aplicación.
- Los triggers PL/pgSQL permiten automatizar lógica crítica de negocio directamente en la BD.
- El análisis EXPLAIN ANALYZE es esencial para identificar y justificar índices con evidencia cuantitativa.
- La separación de usuarios de BD por rol (admin/app/readonly) es una práctica de seguridad fundamental.



Limitaciones identificadas

- Las capturas de EXPLAIN ANALYZE son representativas – no se ejecutaron sobre la BD en producción real.
- El respaldo automático aún requiere configuración de cron en el servidor.
- La auditoría cubre accesos pero no todas las operaciones DML críticas.



Mejoras futuras

- Implementar auditoría DML completa con tabla `auditoria_operaciones`.
- Configurar `pg_cron` para backups automáticos.
- Añadir particionado en tabla `historial_acceso`.

¿Preguntas?

SBVIA – Simulador de Comportamiento Vial con IA

Administración de Bases de Datos – Examen Final 2026

Justyn K. Cruz Pérez | Jefferson M. Umaginga Arévalo | Diego A. Zamora Bumbila

 PostgreSQL 16  Spring Boot 3.2  Angular 17  Redis 7

github.com/keithdrox/SBVIA-AppWeb